QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

libwally-core @ 374df231d054 · Blockstream (libwally) — public repository (passive review)

Organization	QSI
Report date	2026-10-29 05:50 UTC
Report version	1
Audit run	a8a74d72-c13c-47ad-8a96-b74a698831a5 · Document audit · 2026-10-29 05:50 UTC
Analyzer engine	qsi-engine 1.2.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`8797db4762dd52e196cddf2691a7276f78102237791f50ebae31f9746b5cd17d`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

6	0	0	4	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	0
Long-term post-quantum risk	5
Improvement recommendation	1
Insufficient data	0

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-06415	Medium	Quantum-vulnerable ECDSA: transaction and message signing (39 file(s)) confirmed by the analyst	41.8
QSI-2026-06416	Medium	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (6 file(s)) confirmed by the analyst	41.8
QSI-2026-06417	Medium	Quantum-vulnerable ECDH: role not determined (1 file(s)) confirmed by the analyst	40.2
QSI-2026-06418	Medium	Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s)) confirmed by the analyst	40.2
QSI-2026-06414	Info	Quantum-vulnerable ECDSA: role not determined (2 file(s)) confirmed by the analyst	31.2

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 5 confirmed, 1 not applicable, 0 dismissed as false positives.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/ElementsProject/libwally-core @ 374df231d05410a31d605b31c1cc5256581381d7 — evidence links open the lines at this commit.

Supplied inputs

254 files, 6858643 bytes in total, 2 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.2.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
source_code	checked	252	include/wally.hpp: 3375 lines scanned with 16 rule patterns and 123 registry patterns; include/wally_address.h: 376 lines scanned with 16 rule patterns and 123 registry patterns; include/wally_anti_exfil.h: 119 lines scanned with 16 rule patterns and 123 registry patterns
secrets	checked	2	src/swig_python/contrib/aes.py: value(s) at line(s) 12 (test_aes_cbc_ecdh), 15 (test_aes_cbc_ecdh) are declared inside a fixture and treated as test data, although the file itself is not test code; src/swig_python/contrib/aes.py: 2 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name)
dependencies	checked	2	src/wasm_package/package-lock.json: 156 dependencies parsed (156 pinned); src/wasm_package/package.json: 4 dependencies parsed (0 pinned)
osv_advisories	checked	1	156 pinned packages queried at OSV.dev; 3 affected (3 of them only in development or build dependencies)

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 255 (input files: C)

Architecture overview

Static review of the source tree libwally-core at 374df231d05410a31d605b31c1cc5256581381d7. 254 files analysed; of 495 files, 136 are test, fuzzing, benchmark or example code and were left out, 105 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. 132 files come from directories a .gitmodules pins and are read as shipped source rather than skipped as a dependency cache. No requests were sent to the owner's systems.

Overall security posture

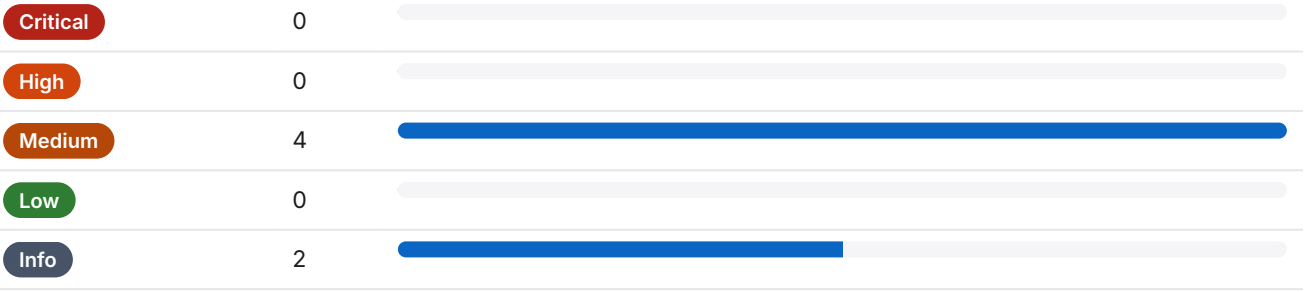
100
Indicative posture index (0–100)

74.1%
Quantum-vulnerable share of inventory

81
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-06415	Quantum-vulnerable ECDSA: transaction and message signing (39 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-06416	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (6 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-06417	Quantum-vulnerable ECDH: role not determined (1 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-06418	Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-06414	Quantum-vulnerable ECDSA: role not determined (2 file(s))	Long-term post-quantum risk	Info	Medium
QSI-2026-06419	7 advisories in 3 development or build-only npm package(s)	Improvement recommendation	Info	Medium

Detailed findings

QSI-2026-06415 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: transaction and message signing (39 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct and central: 39 files, and 50 of the 81 inventory rows are ECDSA. The anchor is real call-site code rather than a header banner — src/sign.c:72 declares a secp256k1_ecdsa_signature and takes a context from secp_ctx(), which is the path every libwally consumer signs through. This is the library Blockstream's Jade vendors into its own tree and Green uses, so these rows are the spending path of real hardware rather than a dependency declaration. Forward-only exposure for an unused key; effectively retroactive for any public key already on chain, which is why address reuse is the behaviour to change before the scheme changes.

135 reference(s) to ECDSA across 39 file(s), e.g. src/anti_exfil.c:72. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:ECDSA:transaction_signing
Data source	src/anti_exfil.c
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-29 05:50 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/anti_exfil.c:72 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/anti_exfil.c · line 72 · SHA-256 38a6257c59bd1576… · File 9a82b2e8a8fb25df…

```
71 | {
72 |     secp256k1_ecdsa_signature sig_secp;
73 |     const secp256k1_context *ctx = secp_ctx();
```

file_excerpt · src/anti_exfil.c · line 87 · SHA-256 9213bc4125f159f4… · File 9a82b2e8a8fb25df…

```
86 |     ok = secp256k1_anti_exfil_sign(ctx, &sig_secp, bytes, priv_key, entropy) &&
87 |          secp256k1_ecdsa_signature_serialize_compact(ctx, bytes_out, &sig_secp);
88 |
```

file_excerpt · src/anti_exfil.c · line 107 · SHA-256 ccc27036c049ec2f… · File 9a82b2e8a8fb25df…

```
  106 |     secp256k1_pubkey pub_secp;
  107 |     secp256k1_ecdsa_signature sig_secp;
  108 |     secp256k1_ecdsa_s2c_opening opening_secp;
```

file_excerpt · src/secp256k1/contrib/lax_der_parsing.c · line 11 · SHA-256 9197354c850cb53b... · File 4da2ecdc0a3b9dbb...

```
   10 |
   11 | int ecdsa_signature_parse_der_lax(const secp256k1_context* ctx, secp256k1_ecdsa_signature*
sig, const unsigned char *input, size_t inputlen) {
   12 |     size_t rpos, rlen, spos, slen;
```

file_excerpt · src/secp256k1/contrib/lax_der_parsing.c · line 19 · SHA-256 234aeaaf6b28e1d2... · File 4da2ecdc0a3b9dbb...

```
   18 |     /* Hack to initialize sig with a correctly-parsed but invalid signature. */
   19 |     secp256k1_ecdsa_signature_parse_compact(ctx, sig, tmpsig);
   20 |
```

file_excerpt · src/secp256k1/contrib/lax_der_parsing.c · line 140 · SHA-256 75ad5d0cba25ca64... · File 4da2ecdc0a3b9dbb...

```
  139 |     if (!overflow) {
  140 |         overflow = !secp256k1_ecdsa_signature_parse_compact(ctx, sig, tmpsig);
  141 |     }
```

file_excerpt · src/secp256k1/contrib/lax_der_parsing.h · line 57 · SHA-256 25af7e242d92af0d... · File 7f3e8ba009f19a5f...

```
   56 |    files (such as tests.c) without the need to set an explicit -I flag,
   57 |    which would be necessary to locate secp256k1.h. */
   58 | #ifndef SECP256K1_H
```

file_excerpt · src/secp256k1/contrib/lax_der_parsing.h · line 59 · SHA-256 4c35a81e99a7eaf5... · File 7f3e8ba009f19a5f...

```
   58 | #ifndef SECP256K1_H
   59 | #include <secp256k1.h>
   60 | #endif
```

file_excerpt · src/secp256k1/contrib/lax_der_parsing.h · line 88 · SHA-256 85efcade66268dd4... · File 7f3e8ba009f19a5f...

```
   87 |     const secp256k1_context* ctx,
   88 |     secp256k1_ecdsa_signature* sig,
   89 |     const unsigned char *input,
```

file_excerpt · src/secp256k1/contrib/lax_der_privatekey_parsing.h · line 34 · SHA-256 163cac24f6b60e79... · File 3420812d623c0f2f...

```
   33 |    files (such as tests.c) without the need to set an explicit -I flag,
   34 |    which would be necessary to locate secp256k1.h. */
   35 | #ifndef SECP256K1_H
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06416 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (6 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct, 7 rows across 7 files, including the real signing call at src/sign.c:387 (secp256k1_schnorrsig_sign32 guarded by a WALLY_EINVAL return) and the extraparams struct at include/secp256k1_schnorrsig.h:82. Taproot spends therefore sit in the same class as ECDSA ones — BIP-340 changed the signature's shape and its malleability, not the hardness assumption underneath it.

37 reference(s) to Schnorr (BIP-340) across 6 file(s), e.g. src/secp256k1/include/secp256k1_schnorrsig.h:82. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:Schnorr (BIP-340):transaction_signing
Data source	src/secp256k1/include/secp256k1_schnorrsig.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-29 05:50 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/secp256k1/include/secp256k1_schnorrsig.h:82 and inspect the quoted lines; the evidence
lists further files.
```

EVIDENCE

file_excerpt · src/secp256k1/include/secp256k1_schnorrsig.h · line 82 · SHA-256 6aae3b20f8f8b503… · File 8185d6c6ea3234ff…

```
  81 |  */
  82 | typedef struct secp256k1_schnorrsig_extraparams {
  83 |     unsigned char magic[4];
```

file_excerpt · src/secp256k1/include/secp256k1_schnorrsig.h · line 86 · SHA-256 43d5f6045ac4e6b2… · File 8185d6c6ea3234ff…

```
   85 |     void *ndata;
   86 | } secp256k1_schnorrsig_extraparams;
   87 |
```

file_excerpt · src/secp256k1/include/secp256k1_schnorrsig.h · line 119 · SHA-256 1e555d327421285c… · File 8185d6c6ea3234ff…

```
  118 |  */
  119 | SECP256K1_API int secp256k1_schnorrsig_sign32(
  120 |     const secp256k1_context *ctx,
```

file_excerpt · src/secp256k1/include/secp256k1_schnorrsig_halfagg.h · line 43 · SHA-256 b3d54ae6f144846b… · File 89669a3ad2c90407…

```
   42 |  */
   43 | SECP256K1_API int secp256k1_schnorrsig_inc_aggregate(
   44 |     const secp256k1_context *ctx,
```

file_excerpt · src/secp256k1/include/secp256k1_schnorrsig_halfagg.h · line 70 · SHA-256 be378be4953e3b3d… · File 89669a3ad2c90407…

```
   69 |  */
   70 | SECP256K1_API int secp256k1_schnorrsig_aggregate(
   71 |     const secp256k1_context *ctx,
```

file_excerpt · src/secp256k1/include/secp256k1_schnorrsig_halfagg.h · line 94 · SHA-256 b10de912dd2b9f3e… · File 89669a3ad2c90407…

```
   93 |  */
   94 | SECP256K1_API SECP256K1_WARN_UNUSED_RESULT int secp256k1_schnorrsig_aggverify(
   95 |     const secp256k1_context *ctx,
```

file_excerpt · src/secp256k1/src/modules/musig/session_impl.h · line 621 · SHA-256 6254128e9a575e14… · File ddcb0d007ebaa3cc…

```
  620 |     secp256k1_musig_nonce_process_internal(ctx, &session_i.fin_nonce_parity, fin_nonce,
&session_i.noncecoef, aggnonce_pts, agg_pk32, msg32);
  621 |     secp256k1_schnorrsig_challenge(secp256k1_get_hash_context(ctx), &session_i.challenge,
fin_nonce, msg32, 32, agg_pk32);
  622 |
```

file_excerpt · src/secp256k1/src/modules/schnorrsig/bench_impl.h · line 33 · SHA-256 fc2c5559f32fc9bd… · File 9655992c7fd21423…

```
   32 |         msg[1] = i >> 8;
   33 |         CHECK(secp256k1_schnorrsig_sign_custom(data->ctx, sig, msg, MSGLEN, data-
>keypairs[i], NULL));
   34 |     }
```

file_excerpt · src/secp256k1/src/modules/schnorrsig/bench_impl.h · line 44 · SHA-256 4f2e9b7b2529431d… · File 9655992c7fd21423…

```
   43 |         CHECK(secp256k1_xonly_pubkey_parse(data->ctx, &pk, data->pk[i]) == 1);
   44 |         CHECK(secp256k1_schnorrsig_verify(data->ctx, data->sigs[i], data->msgs[i], MSGLEN,
&pk));
   45 |     }
```

file_excerpt · src/secp256k1/src/modules/schnorrsig/bench_impl.h · line 80 · SHA-256 0ca8c9b515200145… · File 9655992c7fd21423…

```
79 |         CHECK(secp256k1_keypair_create(data.ctx, keypair, sk));
80 |         CHECK(secp256k1_schnorrsig_sign_custom(data.ctx, sig, msg, MSGLEN, keypair, NULL));
81 |         CHECK(secp256k1_keypair_xonly_pub(data.ctx, &pk, NULL, keypair));
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06417 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDH: role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Three rows, and the only ones here with a retroactive character, so they should not be read as three more signing rows. One of the three is weaker evidence than the others and that is worth saying: it is a printf in a build-information path announcing "ECDH key exchange (optional module)" under #ifdef ENABLE_MODULE_ECDH, which proves the module can be compiled in, not that this build uses it. Where a shared secret derived through ECDH protects stored or recorded data, harvest-now-decrypt-later applies in full, and unlike the signature rows it can be hardened today by wrapping the agreed secret under a hybrid KEM — no consensus change needed. Kept at medium rather than raised, because the evidence for *use* is a conditional compile rather than a call.

4 reference(s) to ECDH across 1 file(s), e.g. src/secp256k1/src/bench.c:24. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDH:general
Data source	src/secp256k1/src/bench.c
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-29 05:50 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/secp256k1/src/bench.c:24 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/secp256k1/src/bench.c · line 24 · SHA-256 6ec91ca104f465f4... · File 832edacf541809e3...

```
23 | #ifdef ENABLE_MODULE_ECDH
24 |     printf("    - ECDH key exchange (optional module)\n");
25 | #endif
```

file_excerpt · src/secp256k1/src/bench.c · line 54 · SHA-256 b21ecd8d53157d4f... · File 832edacf541809e3...

```
53 | #ifdef ENABLE_MODULE_ECDH
54 |     printf("    ecdh              : ECDH key exchange algorithm\n");
55 | #endif
```

file_excerpt · src/secp256k1/src/bench.c · line 68 · SHA-256 ae7392892e7a0840... · File 832edacf541809e3...

```
67 |     printf("    ellswift_keygen   : ElligatorSwift key generation\n");
68 |     printf("    ellswift_ecdh     : ECDH on ElligatorSwift keys\n");
69 | #endif
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06418 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct, 7 rows across 7 files, including the real signing call at src/sign.c:387 (secp256k1_schnorrsig_sign32 guarded by a WALLY_EINVAL return) and the extraparams struct at include/secp256k1_schnorrsig.h:82.

> Taproot spends therefore sit in the same class as ECDSA ones — BIP-340 changed the signature's shape and its malleability, not the hardness assumption underneath it.

2 reference(s) to Schnorr (BIP-340) across 1 file(s), e.g. src/sign.c:387. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:Schnorr (BIP-340):general
Data source	src/sign.c
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-29 05:50 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/sign.c:387 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/sign.c · line 387 · SHA-256 b957a7949930facd… · File 9e52718e9015f768…

```
386 |             ret = WALLY_EINVAL;
387 |         else if (!secp256k1_schnorrsig_sign32(ctx, bytes_out, bytes, &keypair, aux_rand))
388 |             ret = WALLY_ERROR;
```

file_excerpt · src/sign.c · line 456 · SHA-256 b2cc452c48e864fe… · File 9e52718e9015f768…

```
455 |         ok = xpubkey_parse(&xonly_pub, pub_key, pub_key_len);
456 |         ok = ok && secp256k1_schnorrsig_verify(ctx, sig, bytes, bytes_len, &xonly_pub);
457 |         wally_clear(&xonly_pub, sizeof(xonly_pub));
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to a post-quantum algorithm; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06414 · CODE-QV-ASYMMETRIC

Info

Quantum-vulnerable ECDSA: role not determined (2 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Dropped to informational, because these rows are the amalgamation unit rather than a use: src/wally_elements.c and its siblings do #include "src/secp256k1/src/secp256k1.c" and #include "src/secp256k1/src/precompute_ecmult.c" to build the vendored library into one translation unit. The primitive genuinely appears at those lines, so the rows are correct inventory and are kept visible — a reader who checks the files should find what the tool found — but an #include of a .c file is a build arrangement, and the real uses are decided above. automated severity: Medium.

4 reference(s) to ECDSA across 2 file(s), e.g. src/amalgamation/combined.c:34. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDSA:general
Data source	src/amalgamation/combined.c
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.2.0 · 2026-10-29 05:50 UTC
Priority score	31.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/amalgamation/combined.c:34 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/amalgamation/combined.c · line 34 · SHA-256 8cce795a23b61e66… · File ea2c2393b4f2f218…

```
33 | #endif
34 | #include "src/secp256k1/src/secp256k1.c"
35 | #include "src/secp256k1/src/precomputed_ecmult_gen.c"
```

file_excerpt · src/amalgamation/combined.c · line 35 · SHA-256 b0ba42431dd53977… · File ea2c2393b4f2f218…

```
34 | #include "src/secp256k1/src/secp256k1.c"
35 | #include "src/secp256k1/src/precomputed_ecmult_gen.c"
36 | #include "src/secp256k1/src/precomputed_ecmult.c"
```

file_excerpt · src/amalgamation/combined.c · line 36 · SHA-256 9f2414696e322db7… · File ea2c2393b4f2f218…

```
35 | #include "src/secp256k1/src/precomputed_ecmult_gen.c"
36 | #include "src/secp256k1/src/precomputed_ecmult.c"
37 |
```

file_excerpt · src/internal.h · line 33 · SHA-256 a3cf3e2c758ea293… · File f21377ed9eba49d0…

```
32 |
33 | #include <secp256k1.h>
34 | #include <secp256k1_recovery.h>
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-06419 · DEP-ADVISORY-DEV

Info

7 advisories in 3 development or build-only npm package(s)

Improvement recommendation | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): not applicable to this code.** Correct as a single non-runtime summary. The seven advisories are ajv, cross-spawn and fast-uri in src/wasm_package/package-lock.json, all marked dev there: they are the toolchain that builds the optional WebAssembly package, not anything a wallet linking libwally loads. Nothing here reaches a signing path.

OSV.dev lists 7 advisories (1 rated high or critical) for 3 npm package(s) that only development or build tooling pulls in (src/wasm_package/package-lock.json): e.g. ajv@8.17.1 [dev]: GHSA-2g4f-4pwh-qvx6; cross-spawn@7.0.3 [dev]: GHSA-3xgq-45jj-v275; fast-uri@3.1.7 [dev]: GHSA-333x-w4r6-6crp, GHSA-hrr3-gc8f-f4qj, GHSA-w74x-v6f9-j3xg, GHSA-wjr9-j795-jxcr, GHSA-xhq7-xmqj-769h. The manifests mark them as such (devDependencies, "dev": true in package-lock.json, pnpm importers sections, [dev-dependencies]/[build-dependencies] reached through Cargo.lock, or — for yarn.lock, which records no scope at all — package.json reached through the lock graph), so they are not part of what ships.

Affected asset	dependencies:npm:non-runtime
Data source	OSV.dev
Detection method	OSV.dev querybatch lookup of exact versions; scope from devDependencies, package-lock.json dev flags, pnpm importers sections, the Cargo.lock graph

	rooted at the Cargo.toml tables, and — for yarn.lock, which records no scope — the lock graph walked from package.json. Where a runtime requirement cannot be resolved in a lock, that lock is left unclassified rather than risking a package being set aside wrongly.
Analyzer	dependencies 1.2.0 · 2026-10-29 05:50 UTC
Priority score	18.3 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

They do not reach users of the product. They can affect developer machines and CI (supply chain): a compromised or vulnerable tool can tamper with builds.

REPRODUCTION STEPS

```
Look the packages listed in the evidence up at https://osv.dev and check their scope in src/
wasm_package/package-lock.json.
```

EVIDENCE

tool_output · OSV.dev (npm) · src/wasm_package/package-lock.json · SHA-256 baa45062fa297ca1…

```
ajv@8.17.1 [dev]: GHSA-2g4f-4pwh-qvx6
cross-spawn@7.0.3 [dev]: GHSA-3xgq-45jj-v275
fast-uri@3.1.7 [dev]: GHSA-333x-w4r6-6crp, GHSA-hrr3-gc8f-f4qj, GHSA-w74x-v6f9-j3xg, GHSA-wjr9-j795-
jxcr, GHSA-xhq7-xmqj-769h
```

LIMITATIONS OF THE CHECK

Advisory data comes from OSV.dev at the recorded time. Scopes come from the manifests; a registry crate's own build dependencies cannot be told apart in Cargo.lock, so the runtime set may be larger than reality, never smaller.

RECOMMENDATION

Update the tooling during routine maintenance; prioritise packages that run in CI with secrets or that produce release artefacts.

RETEST PROCEDURE

Re-run the dependency scan after updating.

OSV.dev — open-source vulnerability database and API

Evidence

ID	Type	Data source	SHA-256
QSI-2026-06415	file_excerpt	src/anti_exfil.c (line 72)	38a6257c59bd1576fcad77de26a0b8b07e8d637f0457829ebd386
QSI-2026-06415	file_excerpt	src/anti_exfil.c (line 87)	9213bc4125f159f48bc6b0e8d7906dc826e6431c9b90cc8d1e9b9
QSI-2026-06415	file_excerpt	src/anti_exfil.c (line 107)	ccc27036c049ec2fe55b8cab5992c66b4834971552c32740b52ae
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_parsing.c (line 11)	9197354c850cb53bbc475b348c0f97bb3f550706fcced8bf48333
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_parsing.c (line 19)	234aeaaf6b28e1d2ff43854e60a3119e3c3f55229e18841e2dc76
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_parsing.c (line 140)	75ad5d0cba25ca64adb6d971da937879704b4abc8845c17dc1da9
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_parsing.h (line 57)	25af7e242d92af0d3c46ae8d8a48030e247c4bfdeba65068030e6
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_parsing.h (line 59)	4c35a81e99a7eaf5ea7315346f3b8cacb0832f7c4a3b1a3d003ff
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_parsing.h (line 88)	85efcade66268dd4d0b42e8f31b634ea4cd93d9d55cdc2e036715
QSI-2026-06415	file_excerpt	src/secp256k1/contrib/lax_der_privatekey_parsing.h (line 34)	163cac24f6b60e798317e2383fb603da718e728035262dc9759d7
QSI-2026-06416	file_excerpt	src/secp256k1/include/secp256k1_schnorrsig.h (line 82)	6aae3b20f8f8b503597acb97fe491246b6777b5a9660b8a415426
QSI-2026-06416	file_excerpt	src/secp256k1/include/secp256k1_schnorrsig.h (line 86)	43d5f6045ac4e6b23daf11fd6c10022dca16ca76afd775c3ee898
QSI-2026-06416	file_excerpt	src/secp256k1/include/secp256k1_schnorrsig.h (line 119)	1e555d327421285c8a81345e58876f5f2f017571adb2f2168aafc
QSI-2026-06416	file_excerpt	src/secp256k1/include/secp256k1_schnorrsig_halfagg.h (line 43)	b3d54ae6f144846b9769cfe141d006f33bf49fa9572bafb7307c2
QSI-2026-06416	file_excerpt	src/secp256k1/include/secp256k1_schnorrsig_halfagg.h (line 70)	be378be4953e3b3d2e2b5a4ddab26874bc43e6f72392495b730ca
QSI-2026-06416	file_excerpt	src/secp256k1/include/secp256k1_schnorrsig_halfagg.h (line 94)	b10de912dd2b9f3e510c8fe909f0b6db3a25cde95edbbe0af55b1
QSI-2026-06416	file_excerpt	src/secp256k1/src/modules/musig/session_impl.h (line 621)	6254128e9a575e14bb87c68d5f2f12d65a6084dd7de58385e4a27

QSI-2026-06416	file_excerpt	src/secp256k1/src/modules/schnorrsig/bench_impl.h (line 33)	fc2c5559f32fc9bd9a85efef59a46eaf722b1b8e2745b837696cd
QSI-2026-06416	file_excerpt	src/secp256k1/src/modules/schnorrsig/bench_impl.h (line 44)	4f2e9b7b2529431d8190d606bbe0ce49659d6e5a1e0410625d931
QSI-2026-06416	file_excerpt	src/secp256k1/src/modules/schnorrsig/bench_impl.h (line 80)	0ca8c9b51520014576b4c622e33014e8b93cc7c4bd0bc826e9cd4
QSI-2026-06417	file_excerpt	src/secp256k1/src/bench.c (line 24)	6ec91ca104f465f4b861193bee0ed1f6d91c4074fb5085511ca06
QSI-2026-06417	file_excerpt	src/secp256k1/src/bench.c (line 54)	b21ecd8d53157d4f920477f57713fa34b7fe6379df61daa64f27e
QSI-2026-06417	file_excerpt	src/secp256k1/src/bench.c (line 68)	ae7392892e7a0840c3596f3442ca87927d89cc694ec91267d2a73
QSI-2026-06418	file_excerpt	src/sign.c (line 387)	b957a7949930facdb75237e2e6c12333acd87496052f7b47d028b
QSI-2026-06418	file_excerpt	src/sign.c (line 456)	b2cc452c48e864fed16b916f1e68559e9d63deec6656b7f76f359
QSI-2026-06414	file_excerpt	src/amalgamation/combined.c (line 34)	8cce795a23b61e665185923e4058d56ed7e106c55784ee042d2a2
QSI-2026-06414	file_excerpt	src/amalgamation/combined.c (line 35)	b0ba42431dd53977e931a49f059369c0a07f7bb7b7924b65b78a0
QSI-2026-06414	file_excerpt	src/amalgamation/combined.c (line 36)	9f2414696e322db70b097fe19907ea2f68a74d7db34037aad03b8
QSI-2026-06414	file_excerpt	src/internal.h (line 33)	a3cf3e2c758ea293394dc165dc129bd87d56cac6ce0aa935b84a4
QSI-2026-06419	tool_output	OSV.dev (npm) (src/wasm_package/package-lock.json)	baa45062fa297ca1092e6508e54634767572b685acf5a4da92151

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
transaction and message signing	ECDSA (39), Schnorr (BIP-340) (6)	QSI-2026-06415, QSI-2026-06416	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them
role not determined	ECDSA (2), ECDH (1), Schnorr (BIP-340) (1)	QSI-2026-06414, QSI-2026-06417, QSI-2026-06418	Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
weak classically	2

reduced margin (Grover), approved	15
Grover: acceptable to NIST, below ECCG/ANSSI guidance	4
vulnerable (Shor)	60

Detailed findings: QSI-2026-06415, QSI-2026-06416, QSI-2026-06417, QSI-2026-06418, QSI-2026-06414

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDH (3), ECDSA (50), Schnorr (BIP-340) (7)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.
Grover's algorithm — key search and preimages Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	HMAC (7), PBKDF2 (1), RIPEMD-160 (2), SHA-2 (6), scrypt (5)	Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance.
Brassard–Høyer–Tapp — collision search Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	RIPEMD-160 (2), SHA-2 (6)	256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Remediation plan

QSI-2026-06415 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06416 · P3 · 90 days

short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06417 · P3 · 90 days

short-term risk reduction

Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06418 · P3 · 90 days

short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06414 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-06415 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	

	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-06416 · P4 · 90 days root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-06417 · P4 · 90 days root-cause fix

Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH or a hybrid scheme in use.

QSI-2026-06418 · P4 · 90 days root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.

Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-06414 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-06419 · P4 · 90 days

root-cause fix

Update the development and build dependencies listed in the evidence.

Rationale	Eliminates the root cause described in the finding.
Expected outcome	The weakness is no longer present on re-test.
Complexity	low
Dependencies	Change window and owner approval.
Side effects	Tooling upgrades may change build output; compare release artefacts.
Tests before change	Reproduce the finding using the documented steps.
Tests after change	Re-run the retest procedure from the finding.
Closure criteria	The dependency scan reports no advisories for tooling packages, or the remaining ones are recorded.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
None			

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-06415	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06416	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06417	Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06418	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-06414	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-06415	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06416	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06417	Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06418	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-06414	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

P4	QSI-2026-06419	Update the development and build dependencies listed in the evidence.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-06415	Re-run the inventory after refactoring.
QSI-2026-06416	Re-run the inventory after refactoring.
QSI-2026-06417	Re-run the inventory after refactoring.
QSI-2026-06418	Re-run the inventory after refactoring.
QSI-2026-06414	Re-run the inventory after refactoring.
QSI-2026-06419	Re-run the dependency scan after updating.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.1.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.2.0
inventory_document	1.1.0
solidity	1.2.0
source_code	1.3.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.2.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
2. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
3. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
4. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
5. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
6. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
7. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
8. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
9. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
10. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
11. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
12. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
13. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
14. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
15. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
16. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
17. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/ipd (initial public draft (November 2024); proposes deprecating quantum-vulnerable public-key algorithms after 2030 and disallowing them after 2035 — dates may change in the final version)
18. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
19. OSV.dev — open-source vulnerability database and API — Open Source Security Foundation. https://osv.dev/ (the database QSI queries for each exactly pinned version; every advisory id in a finding resolves at https://osv.dev/vulnerability/<id>)
20. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
21. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-

bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)

22. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Quantum status	Classical status	Count	Locations	Examples
RIPEMD-160	RIPEMD-160	weak classically	legacy	2	2	src/ccan/ccan/crypto/ripemd160/ripemd160.c; src/ccan/ccan/crypto/ripemd160/ripemd160.h
HMAC		reduced margin (Grover), approved	acceptable	7	7	include/wally.hpp:146; include/wally_bip32.h:170; src/aes.c:273
PBKDF2		reduced margin (Grover), approved	acceptable	1	1	src/pbkdf2.c
SHA-2	SHA-512	reduced margin (Grover), approved	acceptable	2	2	src/ccan/ccan/crypto/sha512/sha512.c; src/ccan/ccan/crypto/sha512/sha512.h
scrypt		reduced margin (Grover), approved	acceptable	5	5	src/scrypt.c; src/scrypt/crypto_scrypt.c; src/scrypt/crypto_scrypt_smix.c
SHA-2	SHA-256	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable	4	4	src/ccan/ccan/crypto/sha256/sha256.c; src/ccan/ccan/crypto/sha256/sha256.h; src/ccan/ccan/crypto/sha256/sha256_sse4.c
ECDH		vulnerable (Shor)	acceptable	3	3	src/ecdh.c; src/secp256k1/include/secp256k1_ecdh.h; src/secp256k1/src/bench.c:24
ECDSA	secp256k1	vulnerable (Shor)	acceptable	35	35	src/amalgamation/combined.c:34; src/internal.h:33; src/secp256k1/contrib/lax_der_parsing.h:57
ECDSA		vulnerable (Shor)	acceptable	15	15	src/anti_exfil.c:72; src/secp256k1/contrib/lax_der_parsing.c:11; src/secp256k1/contrib/lax_der_parsing.h:88
Schnorr (BIP-340)		vulnerable (Shor)	acceptable	7	7	src/secp256k1/include/secp256k1_schnorrsig.h:82; src/secp256k1/include/secp256k1_schnorrsig_halfagg.h:43; src/secp256k1/src/modules/musig/session_impl.h:621

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

B. JSON

qsi.report/1 · SHA-256 8797db4762dd52e196cddf2691a7276f78102237791f50ebae31f9746b5cd17d

C. Input files

File	Size	SHA-256	Reda
include/wally.hpp	177878	4c71d16bd41091796a4898b24ddc0d30da23414f8741a484a251f4bfee9ad70d	0
include/wally_address.h	14902	7f7c7746344d85a5d4c4dd294eacb28d93f80d7efc306ea47f83adf21d68b9f7	0
include/wally_anti_exfil.h	4298	3b5dc9737f3c2921ef1ef8f8b14254b705fe2df2d30202115ef5bbb85c347833	0
include/wally_bip32.h	18820	810cb696a773ebfc473140d4500466f4755b0874ffe0b4be941524d43684cda9	0
include/wally_bip38.h	4113	21b2a312a50b325c239960e44dbbfc71ed8c0c05db04ac6e4ac85c43f7f3ed6b	0
include/wally_bip39.h	4675	37d73b2b2c5477c56b5f226f8e6a67595f8135e0dbdf9022c5572955b4359068	0
include/wally_bip85.h	2147	e905f637ff408e6e406f9b4ee292be6e4592770c69af6dfbcb46674c63cb88ac	0
include/wally_coinselection.h	1952	7e8ffb7f18e814876726e5dc7338b745b6924ed7318148b6708fa98f17b87193	0
include/wally_core.h	14337	3022b6eee2211127b4b29afc07e9efa6a3b43c620cc5c19c8505a2517c4a33dc	0
include/wally_crypto.h	36912	2181a542b32bbf426de0259360751fa6e821dd45e0f67bc7015533bb08c629c0	0
include/wally_descriptor.h	26151	3a0072ee4e49163f9b2cf63e1d4f1fdb98bc8683560c1c6559cfec3a6f12c82d	0
include/wally_elements.h	30713	12520fe83d0d7a6985a8f89f3483465920a56c70ef8bfe2c5ccea238a7eea498	0
include/wally_map.h	23352	49e8504df27c0a3530f125fb11bf8b6476a9ec4abce40b1b5321121a85cd9e0e	0
include/wally_psbt.h	92379	5f22a3900c2be3fba63b3f0e48fe36509c7db4630ee14a9a6d48675c406a3cc0	0
include/wally_psbt_members.h	28550	02f5d3b6038d0ddc79830686dcfe05f92f5c0fdfd4e3d5974a586d82c1e570c7	0
include/wally_script.h	24869	4ba9f21af3835bd44cf8c9baf740b48d69f7c112d25282bf0154ba1162f74581	0
include/wally_symmetric.h	1403	c592f068170a611a0b4f45610f8af1901f25f7c835898ddf85cf48c02925cc31	0
include/wally_transaction.h	55083	02f09466ce2e7bc8f326f762efde701d02c84334018cb4bc5fa8d909be851007	0
include/wally_transaction_members.h	14257	71199aa909d73759bcb9328885072d810bfabe9cc7f661a3dd0b50597f390872	0
src/address.c	8162	f7faee93c6eec3fa6311ef59e30aee3aa5c38fbedb89567b53787602e5e7ec23	0
src/aes.c	13043	5a9d9d02276f306f18a1afec725d00544d43ce5e991df8f7f9e66f651b37fbda	0
src/amalgamation/combined.c	3370	ea2c2393b4f2f218bed700c4d880fdff8f6491ab98c42de945a14f9c1e265749	0
src/amalgamation/windows_config/config.h	518	1b14c47566007e84ccf825e180841132f485fbfa7cca9157024dde4f4c4b559e	0
src/anti_exfil.c	4182	9a82b2e8a8fb25df7d59ee0108552079c29c7f5e522a9ba9700f4fa10983a612	0
src/base58.h	329	19d52a6c48e467662faf41cbe7bcaf86850dcc79baa1dea43f29031910c8a75d	0

src/base_58.c	10621	2765b178cc94418c37ac520fbbcd7082fb817fa4d2ba484c5abba7971fcaba6e	0
src/base_64.c	2352	fdd2a5e08206baee701ede543348dcf13f6239af39992f5cd653f680461c7bac	0
src/bech32.c	11145	a4a2fddbbf6dfb68752e34814823ac0f486e85fbf3e6a80b180d63c80835843a	0
src/bip32.c	42775	ebe4939b794fa8d447ef1054296842c5e1fa30bf16491dbb5cc5c08151937337	0
src/bip32_int.h	1789	00614706452155f6adcc4699a3bee9537ccb793993ffcd02a69e5c8f16e9aa99	0
src/bip38.c	13750	7ded4248d2176e6f18ef02d556bc138872d191299f460e4ef77f501d4679248f	0
src/bip39.c	7536	0b4ad66758f2f02a9fa5dbaa1daad4094deb8bff5290b612770b8c37d976d61d	0
src/bip85.c	4885	4eeff959873565157d2c8696536bdbb3db402e709e79dfddd7c9b7cb49f01a4a	0
src/blech32.c	11446	6f115f7716d84eb277e942fcb281b55a229b0befc7947f681b6ad861e8383153	0
src/ccan/ccan/base64/base64.c	7742	2660b9eeb6c66e0a8187f8b946471f1d9463bb601bf3780aaf4af901498bdf0f	0
src/ccan/ccan/base64/base64.h	8587	75e1376547570c23b5cb51a58b7eb06e00e22f117f45e3af5510970ea6934795	0
src/ccan/ccan/build_assert/build_assert.h	1228	eeebbbc75d152ab145fee9ea5fb3e914822047a67c487ad2f13d9c724fc9384d	0
src/ccan/ccan/compiler/compiler.h	6208	265ec2faa538935602f4d9527e027aff5d2201630a405c2194212e9e9282cc1a	0
src/ccan/ccan/crypto/ripemd160/ripemd160.c	14205	4d231f50f83073f1a4c0a79f9c3e755e9350b5d5577a2c0e22cf5670dc645a1d	0
src/ccan/ccan/crypto/ripemd160/ripemd160.h	4229	7f7d917d859e20b40c95e94b9a4862eda420dd9caed14b9619d1fcfc0e1d452b	0
src/ccan/ccan/crypto/sha256/sha256.c	12527	286fc60aeaef4c3f23e61f5c43068b39fbfa13336e6f0bee2223eca5865fe93f	0
src/ccan/ccan/crypto/sha256/sha256.h	4392	d68d0b50a1695e3762dc6a18f3e15de2c270c9d533831174c6e2d8dadaa040e7	0
src/ccan/ccan/crypto/sha256/sha256_sse4.c	26723	97974a76fb8ae60ea7b3f10e1f885631b38bd741739bfc29b82f2a9a4a896f32	0
src/ccan/ccan/crypto/sha512/sha512.c	12350	acfe1bc068339cee74a59ec420d11a1161fa3640b3a55e995409a76a3c33e80c	0
src/ccan/ccan/crypto/sha512/sha512.h	3799	2e013c21c38bdcecd5407f21a7cc7edb55d62cdaee0e2b6374241139324b7d1f	0
src/ccan/ccan/endian/endian.h	9168	079dc923af35babc0a19d48920453eac806aa28757809e5e431fb66f3c0511d9	0
src/ccan/ccan/str/hex/hex.c	1240	f19e45cfcd6f9e1a57135d0ab0f7664467a500db35a3377d7efa306bdef0670a	0
src/ccan/ccan/str/hex/hex.h	1907	bb26783b82e632be5a7bc2af185655119124086dbed9ea441eb96d3e30f224dd	0
src/ccan/ccan/tap/tap.c	9251	44ac24b231f4a27d49e3a14bd025394911740291b3ccfa56b0549972a14cdf99	0
src/ccan/ccan/tap/tap.h	8136	0349b6db4d1699703fae8ad17f0d8cd3c585a9eb5f259598d2c1011770914153	0
src/ccan_config.h	1856	fbfef65cb854aff875e982a0f3f49913a7e13e3425ad99fbcfb23ce5a7455526	0
src/coins.c	7481	32a65ff8892db2fa25f1ea43e4c5e4ae5dd2c853a369e771165cbd976ce98584	0
src/cpufeatures/cpu-features.c	40717	60b4e0194012595a58c88fcd4089bc72c4752f8fc829b022b1228a9c39afcc8b	0

src/cpufeatures/cpu-features.h	11746	4e85e43209f0ba184a4bc5d3353d99695d4b7543d436333cf465bd6c7ec0239f	0
src/ctaes/bench.c	4977	237677992b69e7dcc6d4fab66f74e8afa7c999dd3a16598e6acd3c9fdb13125b	0
src/ctaes/ctaes.c	18015	2f8a63e6a9a00da3fcc9c38d53dfbbb650376191689864ed211a36d8184c287b	0
src/ctaes/ctaes.h	1541	bf837cfa914ac30b11cea64d5267065614686d41586c886e84bd4e7e11112af2	0
src/ctest/psbts.h	96356	03ef1939fb98779fe78fd0619ac9af38b4b9f11d13d03ed5a4d5b8958dfc1d2e	0
src/data/wordlists/chinese_simplified.c	66952	0b5757eb83ab683d573f2f33a8f6beb55206f9341399350671a41bc8f97b41d9	0
src/data/wordlists/chinese_traditional.c	66952	e3c7f2043ed23fcc05a2d56c4a0fde20fb8d78e424a9de32d7cad151e8b4ceac	0
src/data/wordlists/english.c	90123	6281a94bbb2f078bf81c5e896ba256bfa647ecf15fba3fc17b56252b96a8eace	0
src/data/wordlists/french.c	108796	456ccc99b4e1deb3c32be31cd1da15fc2c24f68dcdf7bc44f728e92e696d0b2e	0
src/data/wordlists/italian.c	104983	a62efd66b93dba13bd6595c75223c16fba1c64249515745fdf692c1dc1adbe71	0
src/data/wordlists/japanese.c	157532	6e05670b15966f5ed2f9dda1eb65176ef0db0d9627d6722025c8b74f23be6c88	0
src/data/wordlists/spanish.c	94627	2868a35904d7a59825854b1c7aa74b1ceb48260706c841f2c24cd62ad608348a	0
src/descriptor.c	165581	b6e84f5f96656a3516761279b1db5d9fcacd352bad133535e34e1b41c910b218	0
src/ecdh.c	909	8ee530f4d9ae4d9bf59d56be587898902ef61da538b660f427dd222034bcb1f6	0
src/elements.c	45442	f2619a99c89c18b470b3d309f004bfa99994890e5c46df2527fe1f545189cae2	0
src/hex_.c	1814	b2d657a4efdea61b6fc6499384a54e084ece23b08283880d8d275bc36cb02d76	0
src/hmac.c	1029	ec26247e22f48069c0b590cab1df5c9eb6405e174e58bc6ecaca625dcf4c84f9	0
src/hmac.h	945	272f5733e09fcd7bbed99e7090d6dfa45f2804190eb86c9a865b152d71db64a7	0
src/internal.c	18775	fc35a46528e06386282ca70d31fde07b9aaee997a25ca7dcc41b69a04a8b36c4	0
src/internal.h	7781	f21377ed9eba49d06a5664dc4c28437e62d1b2868f4d6cfb75fd844804cd4b65	0
src/map.c	32866	40a415e729cfc3e551ec89728246c01d325376fde2d78e418de9e14f10061475	0
src/mnemonic.c	3004	8a1bb43751a5bfd8702a970eaa6707af12c97df778ea917daedba6f12c75fbba	0
src/mnemonic.h	975	74f5df50a738b55423f5920b0d3d4441c1ceabc680aaac4a63e2454c6004cb13	0
src/pbkdf2.c	889	105f023951c03024c125a18b0f5862fe766f58fd0f4c64c5f389804f2a5bddf6	0
src/psbt.c	243364	e76cac2111ddcdbd1abe201be5f2411cb411156ec84e5848111007e3cfb28809	0
src/psbt_io.h	13779	851894fd3136feb0c911152c290491b5a858cbc454731fb1cebc347a5d8d4b0a	0
src/pullpush.c	7444	777d7edd1d1b649a2dcf32a7db0da3eab02548c4682355420db452bd2d60b6da	0
src/pullpush.h	4514	b1d4ce8e2abed2de4edb1917f4822517e235cfefc31ff238a57e5964f08fb121	0
src/pyexample/anti-exfil.py	660	1a667aaadb838d6e397bb99d1bf6b1cb5a4635c0c03174ef495db6644841b428	0
src/pyexample/liquid/receive-send.py	22276	bc90191a9307600954861a8368f2c4b5a3599a64d10068be4a4064a5a4130a71	0
src/script.c	46749	fb0fe6ddcd942103db5b0d2dc44259d154aea6b1f9f0fc792b6ab49f37932a9f	0

src/script.h	818	1617a0849f8db48baa659756a8df6fd8cda1ad03517959f392aa798f52c29cfa	0
src/script_int.h	3734	bfb7c554290734d0fd4204ebcbad2496f46e4c475bfcfa44b1d364ffa97d06f4	0
src/scrypt/crypto_scrypt.c	7396	033c20dbf6d9353f01f45cf14752afea67811c0fb487b7e2fc086d791c084ee5	0
src/scrypt/crypto_scrypt_smix.c	6241	87c79624c9a5b5d1375dd332734f14758fb87781e5ab4dc3638dabfcb09b0f6b	0
src/scrypt/crypto_scrypt_smix_neon.c	9554	a92a7abf7fcd2cd406acd8a3fa37a16fddf05f05581305d5900daf65a70ecdcd	0
src/scrypt/crypto_scrypt_smix_sse2.c	7101	5a0b698f710cdc38e951d574804fde85b74e67d014da2207a08441de17c616da	0
src/scrypt.c	2523	7912d0bfe719e149ed99ed1dc097e3087e0a9843433aedbeb749c42ac54d8e0c	0
src/secp256k1/contrib/lax_der_parsing.c	3595	4da2ecdc0a3b9dbb9029d188bb0243764a3c2f2324613a688f1ac4d3fc878851	0
src/secp256k1/contrib/lax_der_parsing.h	4230	7f3e8ba009f19a5f2759c9065af02cc8e62351100d69f8ae7e762fa4b65ea675	0
src/secp256k1/contrib/lax_der_privatekey_parsing.c	5112	48352c75520eecd141936c0011601925510cdf60afd3e46e79f0a0547cbc78da	0
src/secp256k1/contrib/lax_der_privatekey_parsing.h	4016	3420812d623c0f2f86debe0535f98eab9b929036bb2fee2de8e3b1adc3e1a0b9	0
src/secp256k1/include/secp256k1.h	44268	e1717d8943d4a28ad0c7dc6c0b19d0afbc40454caa9d1f50d80bb7bb49ed2209	0
src/secp256k1/include/secp256k1_bppp.h	2761	bbd36b1d30ba8d181a84a234a29b91e1af6a6610cb24b7bc61326d12279d0feb	0
src/secp256k1/include/secp256k1_ecdh.h	2560	ad314d14cb79a79d4113862f21e0deccf37962db7811c801dab2899f91dd8222	0
src/secp256k1/include/secp256k1_ecdsa_adaptor.h	7476	5bb2f1c0e958c2380ff3dcc15e5f70d5ecf24517349d25efdedf493882405015	0
src/secp256k1/include/secp256k1_ecdsa_s2c.h	12237	ffcb82928ed6e09cd0e95e15cb3c67d95b1914286cc1fe1408ade0ddf459294e	0
src/secp256k1/include/secp256k1_ellswift.h	9148	620db468b1c13968946e9f404ceb59a3f11c895603bf1dcd3399567d7bf10e99	0
src/secp256k1/include/secp256k1_extrakeys.h	10870	7c3468c9fd192ad152c5cf042cb44938961d752fcf19966123ce6a5e3fd31637	0
src/secp256k1/include/secp256k1_generator.h	10816	1c08ef2a929eb52bff724c2d3d6e8d5a355382d76f9d510c2c4b769e5902ea6d	0
src/secp256k1/include/secp256k1_musig.h	32175	e6e2595005881a64bce69a05cba3e2715e56a7a06398c029f736cc52bcfedc38	0
src/secp256k1/include/secp256k1_preallocated.h	5966	b7283d4cc9c13bb2572dc92e69cea76f3270403baf5748ea4d91a1064b951b85	0
src/secp256k1/include/secp256k1_rangeproof.h	11792	32ac0541545b6d8b562cd7e41043807fd06aed80dbdd89d820538f04e3b4e305	0

src/secp256k1/include/secp256k1_recovery.h	5250	e00678a1108acb36c6631a1e290cf33813b0ed5965d23226367008f58abb27f2	0
src/secp256k1/include/secp256k1_schnorrsig.h	8177	8185d6c6ea3234fff05f38fad4b1b6f48f0574ab599dfc87297ab9bca0abfca5	0
src/secp256k1/include/secp256k1_schnorrsig_halfagg.h	4908	89669a3ad2c904072c1177e7b783996af5d72635bb7590e02284e776cc4e085f	0
src/secp256k1/include/secp256k1_surjectionproof.h	12407	f6b245fe8234309715225fb6a3fdf3a4d20a4f5a7f395c1686debae189af07a0	0
src/secp256k1/include/secp256k1_whitelist.h	6241	2e060720e5f8bdef9b99021a3c1d45a610219f4026d1421c86a8112a0e5e7d66	0
src/secp256k1/src/assumptions.h	4667	cc875f356cc8f7f272def71f7d7261e44cc27a18a5b668cd8192c84d238e4e68	0
src/secp256k1/src/bench.c	10190	832edacf541809e3cbc59a4e698d5e33a9388a7dddd55662936244cd106fccce	0
src/secp256k1/src/bench.h	4777	12fafa36167bb38d28698536cdb34bdbcf8c972eed4d3cf69e71796972b880d1	0
src/secp256k1/src/bench_bppp.c	1102	7c17a6b0fada57db18cc68b1ec183f1e6346983903ee1da4f894acb09bfab6e2	0
src/secp256k1/src/bench_ecmult.c	15684	7daaf2e676b42f08a509b846139e90f9f04da94aaeda62a64ce38f64e45884a4	0
src/secp256k1/src/bench_generator.c	2001	6396798ee1f4a018d4ebd9e962b6c289f4a1928a870c20918285500f7972bf61	0
src/secp256k1/src/bench_internal.c	19230	6fb683c1d9dd79261e9a519c033ee6934687f18bcdad0327f570769d57fe0e42	0
src/secp256k1/src/bench_rangeproof.c	2349	6c24fa0c6c12242da3b2629742e0baab65b24967d173fb83e94e2ce1847d8a10	0
src/secp256k1/src/bench_whitelist.c	4089	8f77ebf85792d8270790e1c0689d24896c2da3d7ce6f35c6dffec2909ddc4082	0
src/secp256k1/src/checkmem.h	5355	f367089bffa53b8998047de2f718f75d627dad549a1e5dc2a9ce9ab9f581ffe8	0
src/secp256k1/src/eccommit.h	2198	3b04b7fdae817984c26ce995fad4f3f6d72d5c05b9d3f78d5991c531a5e99eed	0
src/secp256k1/src/eccommit_impl.h	3165	6c71a9cba341b7782302741603db3ae2179b699ce2e7670cb03a749b94d114ca	0
src/secp256k1/src/ecdsa.h	1164	f97753416b0971133ad9d8acf9af02259931d0285a0333a9753201a353070ef6	0
src/secp256k1/src/ecdsa_impl.h	10562	d396c1c758db5dffdc3044f00895bc1206f797003ea7f17004628c66b09e1c99	0
src/secp256k1/src/eckey.h	1449	cf511599285faecd27ace6191eca3945d85bc852b11076a1b1727759a6b03056	0
src/secp256k1/src/eckey_impl.h	3439	4d327c593520c3f60f7b86d407df0870c26cf58037e4156ba8f92258304fd6b6	0
src/secp256k1/src/ecmult.h	2768	1e87f35efb7e7e5b6ea1d6ec22bbaeab33ccd4eacc41a9092ee518ece9237ced	0
src/secp256k1/src/ecmult_compute_table.h	1073	5a8ce69b18d777c3ad608ef173fd6a47c75b9c37bc6801f19ca4c44eb4b9f8f6	0
src/secp256k1/src/ecmult_compute_table_impl.h	1916	6082339f616cd5a628300df24e66353c1374a117e24ddd474fe6151dbc620d60	0
src/secp256k1/src/ecmult_const.h	1354	4705e7baabcbf8fd1853f8cef45a7f6a3075b98925d91cc209afda2c956942ba	0

src/secp256k1/src/ecmult_const_impl.h	19584	07238bcf17c3292bb8198f7f37baf068bdc9a6a57f843c439e220eb08f400607	0
src/secp256k1/src/ecmult_gen.h	5458	38e1457b6e8ba8f488e5a0e19fb625df97603c0c3c7e03a34bad22026f365fd3	0
src/secp256k1/src/ecmult_gen_compute_table.h	672	51eb54b0cb7b49b25cf751efb2799322d73c97bc9bdd2a263feff2d2d6eeff9f	0
src/secp256k1/src/ecmult_gen_compute_table_impl.h	4705	9ae524743bb5871bff7e41e48b61218336b1919769132aafa944283ef1bb88f2	0
src/secp256k1/src/ecmult_gen_impl.h	16504	568bb11a3ecbbf116f13009b9e6de78c23fce2521f5961db1329203b5d0ab219	0
src/secp256k1/src/ecmult_impl.h	34325	ce70d26cb987c9bb2a2477389459437fc2831546c9c14404b58cf7ed07f32cc1	0
src/secp256k1/src/field.h	15119	a031e8655f38464b47aedec8afd73b3320da565fb50d163ae4f3c0ee247e3444	0
src/secp256k1/src/field_10x26.h	2338	ba9a43b62e7e70e6e18af2787bd01dfc3a57fea527cf8ad0333194211ac7d7de	0
src/secp256k1/src/field_10x26_impl.h	52106	6f209a533fd96e73f4c92b101bd3ca46a2f7553967ba7f44650a68ccbf4fbcf6	0
src/secp256k1/src/field_5x52.h	2380	cbaa91c253ba84d99efcd334c7a39906c164d5edf3cde81200a44a606205cdce	0
src/secp256k1/src/field_5x52_impl.h	19360	8104a1b630635c9cc89628142c9fa7afc6f31a888fe50c8ebb2bdf69ca208975	0
src/secp256k1/src/field_5x52_int128_impl.h	11341	ee6d585731507c51dc8acd819b66556cf0cd804da1a4b6072a79c17eafbc940a	0
src/secp256k1/src/field_impl.h	14052	33cb488ee009334a7a2ac52a7eabe39795bd184da0238e7acf73ae11374643b3	0
src/secp256k1/src/group.h	12243	d5443ba9dc805fb378d6dcdab02bd616ab34409b7b91ceb342ff4cee12adcf7f	0
src/secp256k1/src/group_impl.h	35125	663da90fb18b6fc8dace045ceb28ea8fae6e1c68081c209cf0c7dc954398f30d	0
src/secp256k1/src/hash.h	2625	a7f39f2ed170d70841eff7d43d9c480dfda6cb2e8ee25e7e04bf7433c27193af	0
src/secp256k1/src/hash_impl.h	15448	70247571d95e3c8824d6ca2e90956f6941c196a42d860083040690aaf9ccf6cc	0
src/secp256k1/src/hsort.h	1685	220cb669835d34c252598ecff860c93edda337a9f8c2fc07b2b6cead7d48b284	0
src/secp256k1/src/hsort_impl.h	4798	2861489c01b927b27d99efe173799a812434748b4addd30d102187ce4f3b66a9	0
src/secp256k1/src/int128.h	3622	89ac5d91d64f857038d063dd70eec4b29b4f32aa218c790fe049137c347186cb	0
src/secp256k1/src/int128_impl.h	376	5683c02dfc495639da259d80624711c1c449bbaee6186729e399e713a28b4719	0
src/secp256k1/src/int128_native.h	581	b4c52c7ff091083f5eca9879b98641c7a38356a3c5321cbcc22ae1573e7711e1	0
src/secp256k1/src/int128_native_impl.h	2881	ecaafefadbde3db034d4f44678096833d2d113fd21b4ff7044095b6153ec03a8	0
src/secp256k1/src/int128_struct.h	229	0733b3a65719a1e9a8fb3564c71a38f2f31ca3522f40087d536dba90361b66e7	0
src/secp256k1/src/int128_struct_impl.h	7884	0b4e2200327f5357078ba43b0e0514c58c2b1eb217b87ebc14d95efb913b5be2	0
src/secp256k1/src/modinv32.h	1865	774887ccd462b145434d1d6b9ae9455aa3618e791544a2b61267e18931d5010d	0
src/secp256k1/src/modinv32_impl.h	33492	befe02af7099a871677c7b05a81302dff5a4dc1d59f19b20bcb58cc28e096112	0

src/secp256k1/src/modinv64.h	1969	b71cf177d768beac28fcb4f0ef545aafaad16ad0f275c70ee4c60b5f7549ac41	0
src/secp256k1/src/modinv64_impl.h	37653	1254dc7ea941fc4d4ef0a85eed80db58d9e2647a8b8d19238d6afc048d63764d	0
src/secp256k1/src/modules/bppp/bppp_norm_product_impl.h	20378	13b376aadaf6ad496b82de06e57ce781c8dd591670405a50b41beb06d8b71614	0
src/secp256k1/src/modules/bppp/bppp_transcript_impl.h	1552	bc8b8afcf3e985bdb355e715c2f56f109e2fb5a109f393fb0359db17b8aafbe4	0
src/secp256k1/src/modules/bppp/bppp_util.h	2794	fb22fd546df224866adc6c756f76170b4aa2575cdda6872ee56f32af8079db95	0
src/secp256k1/src/modules/bppp/main.h	403	20d3c6914baf5513f17d02787e51a97ecaa1670202ea5ca15bf7b09a35b0aa80	0
src/secp256k1/src/modules/bppp/main_impl.h	3583	938885438c1a41562294d15e068e9a7dac58972113e7e4945cd945f196dd6e5b	0
src/secp256k1/src/modules/ecdh/bench_impl.h	1761	3929ead352b3b6425f0fdf94a1b2268d124cc0176929ae6ea3b4fa82abac0bbf	0
src/secp256k1/src/modules/ecdh/main_impl.h	2929	3cf3b168e8048cd7b03c9b4d56d072214ef859a31bc970e5a16c2cbeba8fc4eb	0
src/secp256k1/src/modules/ecdsa_adaptor/dleq_impl.h	6275	d5973e606b4225b4cbce6ea02496385f3a8e9c221aa3bcf6f62c108357baf0de	0
src/secp256k1/src/modules/ecdsa_adaptor/main_impl.h	15004	157bf8e45ad426f56583b7def22d91418c0d4bec281b5ab9cd5fce3158a5a238	0
src/secp256k1/src/modules/ecdsa_s2c/main_impl.h	8784	20bea71274c43c26093e5e00a22a9a0ac97eca2330291e2b2fe9f9864cbb3e4b	0
src/secp256k1/src/modules/ellswift/bench_impl.h	4399	8973d64bda27eb72a4807aae665a1a2932d624370f9d40f8833f7c8f03b05564	0
src/secp256k1/src/modules/ellswift/main_impl.h	27602	8deffa7b37a0602be2b87d0378419410e6cc9683ebcf3fb033d2eed56c4f8416	0
src/secp256k1/src/modules/extrakeys/main_impl.h	9961	a58762afb6a494aaec984d2e65adacf1125be5e55ad2904af0f26e490cc0d3b4	0
src/secp256k1/src/modules/generator/main_impl.h	17287	756c2aea7b7221d6b523ae17d367a6e819b67f87485987b2bfff71ed75e2e785	0
src/secp256k1/src/modules/generator/pedersen.h	904	6529271ae3f7e786df540f650164e66cae35478157b5bba6fc885cde61d3e81f	0
src/secp256k1/src/modules/generator/pedersen_impl.h	1729	1cfde5758fb1d3cbe4a2b8f6e9f6ef00eeb75f6895810dadf213bffc4553297a	0
src/secp256k1/src/modules/musig/adaptor_impl.h	3359	e22baaaa040a80c84f0cd72a8aef3b6ba4c260a926d4039bebb2618d54b78010	0
src/secp256k1/src/modules/musig/keyagg.h	1324	5349443b84525a85434ff8f92a1e7542c0982b1b783a682613026203de4148dd	0
src/secp256k1/src/modules/musig/keyagg_impl.h	11171	fa67be67a1cdb0ad05fc6fd5fce158731ea499a18f1b616bd17fe3cf19dead87	0

src/secp256k1/src/modules/musig/main_impl.h	452	ea8d83ca27ceafc2d119c82d62975840302871978dd7a0309f8e3bcc132ed48e	0
src/secp256k1/src/modules/musig/session.h	861	c256c3742a68849520569845640f152a22a4a8c21a96040cf0c1054516983980	0
src/secp256k1/src/modules/musig/session_impl.h	31530	ddcb0d007ebaa3ccf9611de3671c8551ef93e7f33b588d27156d42a95bc46ede	0
src/secp256k1/src/modules/musig/vectors.h	37797	c7cd9dc3804d864d17e43c5b00335a405a3bc4a30861838ac4599a213231e132	0
src/secp256k1/src/modules/rangeproof/borromean.h	1167	204588b38d00b1db3765ab4b2c25e01f5afb5f5920a380cfeca381ace064377f	0
src/secp256k1/src/modules/rangeproof/borromean_impl.h	7977	4e1e6b782851715a87440d62bbfbf7f6db6d292dd77a00a43e5605233aeada6e	0
src/secp256k1/src/modules/rangeproof/main_impl.h	5051	5403c2086d4b81f9e85a91f2290cafd3234e111fae7adbd065a2f5eb78adae82	0
src/secp256k1/src/modules/rangeproof/rangeproof.h	956	a8d6285ada07204cc46dab26087b3c61b732c3b37cbcbbbcb793a8705f0ff80f	0
src/secp256k1/src/modules/rangeproof/rangeproof_impl.h	26166	94685312146bc3ac7ead5f4a29cddf7d9c51bdb2165b1a27af5ba2a0a6814137	0
src/secp256k1/src/modules/recovery/bench_impl.h	2320	20d13192a4aabaa52ac4c2a2da772d8aafef00773051836b25faa2816226b386	0
src/secp256k1/src/modules/recovery/main_impl.h	6153	93fb7455adaa364a3c1f8acef82ba409b88a49f688dfe23ab8555510fc614c96	0
src/secp256k1/src/modules/schnorrsig/bench_impl.h	3795	9655992c7fd214231623716a738a1928e24a86b4bcdd909f331712df800a1abc	0
src/secp256k1/src/modules/schnorrsig/main_impl.h	10970	138c330969201f4d7ff5eb512715b7e05665c3ab4f6d5d9b4172a41aa7c80164	0
src/secp256k1/src/modules/schnorrsig_halfagg/main_impl.h	7991	ba7c408bf639159630920933e29fb97be7be5ebcbf978716f8c0b839ed4e6ffc	0
src/secp256k1/src/modules/surjection/main_impl.h	16513	5a8dc46515e1a885f221fc8c036ef08c219a2d211fd403b86b482f4ef15904fb	0
src/secp256k1/src/modules/surjection/surjection.h	1066	9ed963634320ce5779dfadc4543cea15493722b7b70046692b520e8d78fde2de	0
src/secp256k1/src/modules/surjection/surjection_impl.h	3758	96034d371d002e572e6e3e7b79aadf3652f13746cc17713846c62099ac7ed2c7	0
src/secp256k1/src/modules/whitelist/main_impl.h	6398	a6cb604aa66933d4c5bc1b8e9942eb2c8ea8fc5efb47743dbdd03f07dff1ff0b	0
src/secp256k1/src/modules/whitelist/whitelist_impl.h	4653	6d5bfd9b6dde5d1186c0c886f3a8935b8f30c137b447c19226796b539e2adf9c	0
src/secp256k1/src/precompute_ecmult.c	4017	8085ea86390eab2afeeec8b4f952036492fb81606dcd7bb038f940924b1b8400	0

src/secp256k1/src/precompute_ecmult_gen.c	3864	23d1dadb40296d2752d024678e405b18d5c904fbd61d74598df3c8005ba64d2c	0
src/secp256k1/src/precomputed_ecmult.c	2409168	1fba19c482de09f566ed9e7bf6a526e4edb59a1cf38218e3ab413000a0e0deda	0
src/secp256k1/src/precomputed_ecmult.h	1585	d792b8a49759deff5f1e4092558fa9ae11d24e02695e44bee63a8f0ba8e18503	0
src/secp256k1/src/precomputed_ecmult_gen.c	259715	c0a7af0884b7b07962be67e9da674e0d75a8d412d116153b8eae2a9e5527d401	0
src/secp256k1/src/precomputed_ecmult_gen.h	973	8f514bed441bccf7e34ff05ff49d3a44e9395519a8fdcf176e6aeab7b5dbd5fc	0
src/secp256k1/src/scalar.h	5729	87fea4d072b4db4a093527e7df75b654e66cc89202c07a7f7d20d44c12cb0f65	0
src/secp256k1/src/scalar_4x64.h	801	42aed583b28bda7982c54a6f63e50c0ae5f9c27d5f66d92ec6bf6c90e765735a	0
src/secp256k1/src/scalar_4x64_impl.h	38294	61d0fb4bf51c36819bb778479e7735605d64f8a23073c53afbc3c1eb14f4080d	0
src/secp256k1/src/scalar_8x32.h	725	0614cea113aea88a47ef3b2ba8167722e850d7b2a696a4f16b03ec9a320ef1d6	0
src/secp256k1/src/scalar_8x32_impl.h	32526	2fe7d67f3218e9fd60fc959a302571e7fbed0e302ed53ec8a3d797f629f6650d	0
src/secp256k1/src/scalar_impl.h	12113	6ba654bd4edc0349d1d7f1dfc25ba2d73dd9f2ecdca8078e384bf7b7837d663f	0
src/secp256k1/src/scalar_low.h	1135	3cab20db10224ff85ee7c6d975d2968c1370ed8e96ea26b94650f6e2bac31ac4	0
src/secp256k1/src/scalar_low_impl.h	6258	c7c73a26dae95abd861b7927e5c0a1add5dd7efb20de69b3e62ff24987170206	0
src/secp256k1/src/scratch.h	2224	4af5e1ff665c0418bcc5b1c2de954e7b377f9bdfdd4309e0296938918fac11ef	0
src/secp256k1/src/scratch_impl.h	3677	294a55d429fa4cc9dec46ade345bbb2bfdff4055eb1cbf1ba6168a23e5e74db6	0
src/secp256k1/src/secp256k1.c	34307	825b8e74f872f89ebd4d7bd3e404a889cf9e0efb1bf6f3441a502b9e15de2a31	0
src/secp256k1/src/selftest.h	1494	d695cd5a7369b372285ba0371edb6bebdb3dd79b719af336c714a3013adba6d3	0
src/secp256k1/src/testrand.h	1950	bfe1b9a78209aa90d5652a83efccf633bbd874ed72dfe523f2ff6125f97ceba1	0
src/secp256k1/src/testrand_impl.h	5786	ac12496cdb04f5432b14dbb60ab464317669532fffa5b1de327e21075f01dbb6	0
src/secp256k1/src/testutil.h	4945	5a780fc16fcd8549ea4774d15beb2d36c6ad9a2ca7180dd95f84199a2863d1ff	0
src/secp256k1/src/util.h	16482	a203e19a64dc197ac051aadad15163515fb013ea37dfb8d241929bea4cd09ba7	0
src/secp256k1/src/util_local_visibility.h	402	d47d4a58f40157ab2a298151f366c5ebec8f7a1e7aaba88e36b530b1c01584ca	0
src/secp256k1/tools/symbol-check.py	2416	1706069df728c1a2ac1d2578897076713c16eed73315be6c83ad9690007bca70	0
src/secp256k1/tools/wycheproof_utils.py	427	9f2c2368953eeb3c9b7c3b31a807cde76b3106df93fbb975993638d5eb1a1e8a	0
src/sign.c	28672	9e52718e9015f768784b457a4401d17c9d520b772d323ea87d6c8c69dbbac168	0
src/swig_python/contrib/aes.py	2010	a908a69ec88322237a640666b30549f7ffbcfbb6f1161fc27b7910e81718f28f	2

src/swig_python/contrib/bip32.py	2642	3f656b4c9cf631614cc98b3bd4fb86e83d25ec396af121a26da6f76715b1e888	0
src/swig_python/contrib/coinselection.py	865	e984a050ea05ed45367d0b9beeb21b25a039575b88fb56cfe9dda90cf30ded75	0
src/swig_python/contrib/descriptor.py	1238	293f13173b5700a5670f3fa59c09320769473ea577340fa1cbf4cc5f2a31b1d1	0
src/swig_python/contrib/elements_tx.py	8121	f7ed910ed53a4ba8f0cbc4cf65f2f998b8753d29f3b732716a2561432e4c0d4e	0
src/swig_python/contrib/mnemonic.py	1913	0affbe17869a1d77bc388af7b61d71b7eadde358f9804fb485c40ace73390f95	0
src/swig_python/contrib/psbt.py	42984	de4eede31606e8bf3e7bf885648765a1214fc951bc83ad5e42f8732e7827e41d	0
src/swig_python/contrib/reconcile_sigs.py	2854	0160b8f879594d0c33c9e08e0e0699e3d65a797d504383c3d2b42238d503b087	0
src/swig_python/contrib/sha.py	2019	3a36b14987e4051ba544d89424b76f37bede9b87791f33b27bbb046dd10f813e	0
src/swig_python/contrib/signmessage.py	1589	62c57fe76135de42dcd48c08bd33a0165ec566ad5610e58e9fe636723ecc3414	0
src/swig_python/contrib/tx.py	4916	23ea1bea593ab10be4a4ade0c979ddf184b577016f802a49aded92238c0f8ead	0
src/symmetric.c	1767	746f5c8d602544e0db731e54a287703536d6b462747001dff5122c2b10cd458f	0
src/transaction.c	124444	c9e1504610735267d7048c56d49da1935a29c128eeb10d9d0b7416a9eae23619	0
src/tx_io.c	41304	0b0c24cd86fc8aca60d6753d7fa837fb1ab1f3a64c122dd3948f9ce8cd37ad38	0
src/tx_io.h	1145	2bc60074beb6cf037355e7ccbef1a57b446931babe9c4bf66b7b1fd91a6be4bb	0
src/wasm_package/package-lock.json	127254	1c4bd9a6277a0372be350105ef2acf9b0f3700d67253e711079c857e5b8aae66	0
src/wasm_package/package.json	608	f1c0400fbd6b50d7866ede0d6c960bacfaefdc9fd9d0f9d76a422ca3353bd424	0
src/wasm_package/src/browser/assert.js	321	67e799d7b585943af24ff3d31d5d25b556d3f94812d2219d3eb703a472f4b82c	0
src/wasm_package/src/browser/crypto.js	114	1a0550b75fc50491d16e6564434529adaa5e77b60cc8abae1dade004b0d71e2d	0
src/wasm_package/src/const.js	18836	294fe5332d0323913a1f8eafc34d15e3ac8d07323a0ef239d2ee715367880a7e	0
src/wasm_package/src/core.js	11103	cdd31896d2aff07dfa172e62b63c0d6c7d68f6fe69eaead859a1fa13297eb3ad	0
src/wasm_package/src/exports.js	110	95b7045e0888c44564dbdad98a4a97d10162767c28ace45a58a31a5b3f1fbf8e	0
src/wasm_package/src/functions.js	119691	54900f745051f0b068d7c08a32294c505a2249b284f7ce9b8712363826d7cddf	0
src/wasm_package/src/index.d.ts	96119	68db6001469e4e5f3e41f81f88b3e79fa6f7ad42f904321a14dd768709d1ce19	0
src/wasm_package/src/index.js	644	e604bf7297be553a02b5a716288bb479d6b39571d3302aaf7c4da52c3a034acc	0
src/wasm_package/src/util.js	1217	34e03d693697456630eb3a9db4d8f253271b6536e0fbb7d6deb725081d715ffd	0
src/wasm_package/webpack.config.js	458	9a861d818ed3d7e3b9cae555eb82409c4acc722153ca7d8a0f1089e389e61304	0

src/wif.c	5335	2e01925997bcd34600c7e6e7e8831120b157fd13707a4897f411afc3c260d084	0
src/wordlist.c	2726	d3f630d36f8a7763a67daa88b7652ed40f77540302597574b9e059d234ef5ec2	0
src/wordlist.h	1678	7e8993598f450d95843cd4217f2d4fcfccf349a7464a8bc9dc0457c79a84f5d7	0